1. Edge device and Cloud Cooperative Data Ingestion System for Radar Data

1.1 Overview

A fully operational edge–cloud data ingestion system designed for multi-modal transport environments was developed by IFAG to explore the topic of edge-cloud continuum data ingestion and processing. The system supports local preprocessing, edge inference, gateway-level fusion, and cloud monitoring of heterogeneous data streams. By distributing computation between devices and the cloud, it provides timely responsiveness, reduces communication demands, and enables scalable cloud-based monitoring. The result of task 3.2 supports advanced transport monitoring, fleet management, and intelligent mobility services.

1.1.1 System Architecture

The system follows a data–edge–gateway–cloud hierarchy, as shown in Figure 1:

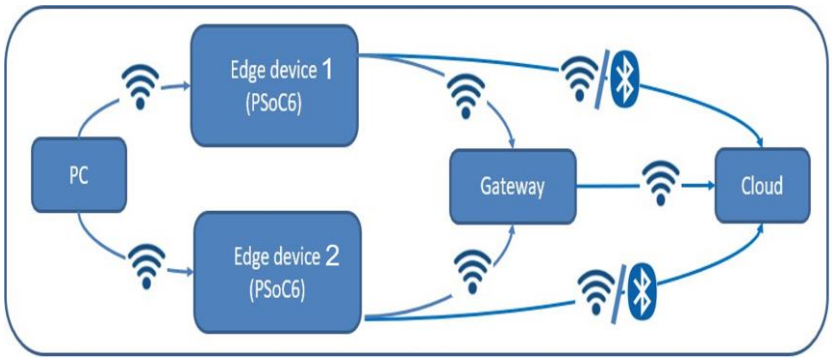

Figure 1: The main structure of the data ingestion system.

Data flow and processing pipeline of the proposed system, illustrating the ingestion of radar point cloud frames, embedded preprocessing, inference, visualization, and cloud reporting, as shown in Figure 2:

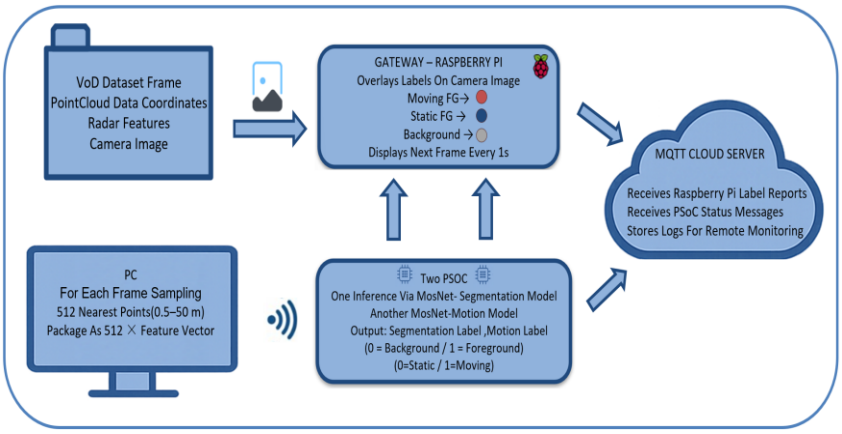

Figure 2: Dataflow pipeline.

The main systems are:

1. Data Preprocessing on a PC serving as a simulated data source or sensor
    a. Radar point clouds and camera images are preprocessed and downsampled on a PC. The preprocessing pipeline is implemented in Python, making use of libraries including NumPy, OpenCV, and scikit-learn for matrix operations, data resizing, and noise reduction.
    b. This preprocessing step reduces the data volume, lowers bandwidth requirements, and ensures compatibility with downstream edge devices before Wi-Fi transmission via socket-based communication.
2. Edge Inference on Embedded Devices
    a. Two PSoC6 microcontrollers serve as edge devices, each deployed with lightweight machine learning models compiled into C/C++ code. Inference is executed using TensorFlow Lite for Microcontrollers

(TFLM), which provides optimized kernels for devices with strict RAM and Flash constraints.
Device 1 runs a Motion Object Segmentation (MOS) model, distinguishing moving radar points from static ones.
Device 2 performs foreground–background segmentation, isolating relevant elements in the radar data.

b. The models are quantized to 8-bit integer precision (int8) to minimize memory footprint and are executed through CMSIS-NN kernels for accelerated low-level computation. This modular division allows parallelized inference under limited compute and memory resources.

3. Gateway-Level Fusion and Visualization
   a. A Raspberry Pi 4 functions as the system gateway. Results from the two PSoC6 devices are transmitted via Wi-Fi and aggregated at the gateway.
   b. Data fusion logic, implemented in Python, merges two segmentation results with camera imagery. For real-time visualization, Matplotlib and OpenCV are applied to overlay radar-derived object segmentation on live video feeds.
4. Cloud Ingestion and Monitoring
   a. The edge device status messages (e.g., “receiving,” “sending”) and the gateway fusion results are published to the cloud layer over Wi-Fi under the MQTT protocol (Message Queuing Telemetry Transport), facilitated by the paho-mqtt Python library.
   b. In the cloud environment, a Python-based MQTT broker (Eclipse Mosquitto) handles message routing.
   c. Persistent data storage is supported by Influx DB, while data security is maintained through TLS certificates and device authentication at the MQTT layer.

1.1.2 Evaluation Metrics

To assess the effectiveness and robustness of the edge–cloud data ingestion system, a set of evaluation metrics is defined across different system layers:

- Ingestion performance: End-to-end latency (p50/p95), bandwidth usage, packet loss rate.
- Edge inference: Inference time per frame, peak RAM usage, model accuracy under down-sampling.
- Fusion quality: Stability of moving/static object detection, IoU/F1 for segmentation performance.
- Cloud observability: MQTT connection uptime, reconnection count, security certificate validity.

The data ingestion system demonstrates how distributed processing enhances efficiency, responsiveness, and scalability in transport-related applications. Key results include:

- End-to-End Data Ingestion Pipeline: Established a seamless flow from raw data to cloud storage, with standardized data interfaces.
- Real-Time Edge Intelligence: Deployed two lightweight AI models on PSoC6 microcontrollers (MCUs), achieving low-latency processing under constrained resources.
- Decision-Level Fusion: Demonstrated integration of dual edge results at the Raspberry Pi for high-level information fusion.
- Visualization and Monitoring: Provided fused outputs aligned with camera imagery and cloud-based dashboards for remote oversight.
- **Edge and Cloud Ingestion Cooperation：**
  The system uses edge ingestion for real-time motion and foreground/background analysis, while cloud ingestion keeps lightweight logs. This cooperation provides fast local intelligence and scalable global visibility, compensating for the limitations of each method.